

PID 算法

应用笔记

编号：EE-202305-CM-APP

版本：1.00

闲鱼：电子工程师的杂货铺

作者：	电子工程师的杂货铺	**日期：**	

更新记录

日期	版本	变更说明	作者
2023-05-06	0.1	创建	

闲鱼：电子工程师的杂货铺

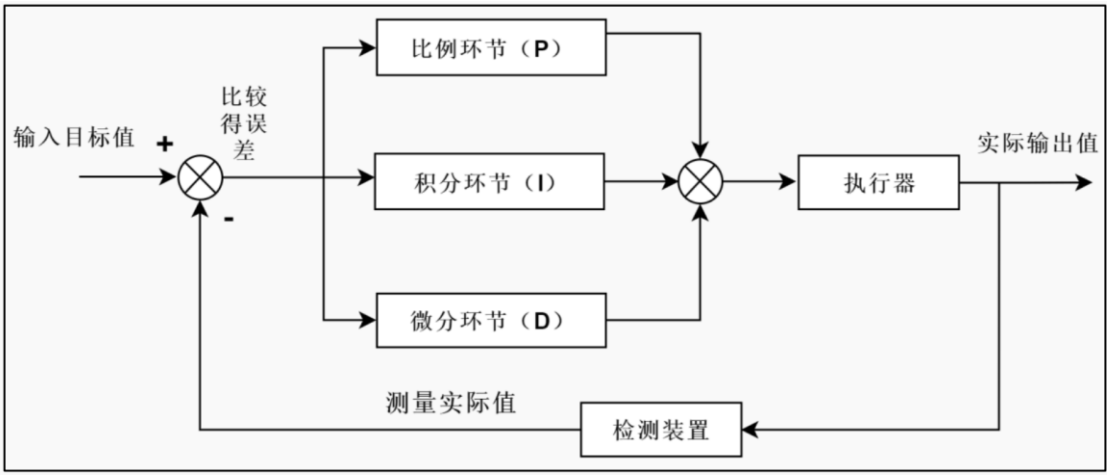
目　录

一、算法模型 4
二、算法应用场景 5
三、算法讲解 6
1）比例环 6
2）积分环 8
3）微分环 10
4）PID 算法 11
1）位置式 PID 公式 11
2）增量式 PID 公式 11
3）两种 PID 公式的优缺点 11
4）积分饱和 12
四、参数整定方法 13
1）试凑法 13
2）临界比例法 13
3）一般调节法 14
五、软件编写 15
1）创建.c、.h 文件，并添加路径 15
2）创建 PID 数据结构体 15
3）算法初始化函数 16
3）算法实现（增量式） 16
6）算法实现（位置式） 17
六、仿真软件使用 18

闲鱼：电子工程师的杂货铺

一、算法模型

PID 算法是闭环控制系统中常用的算法， PID 分别是 Proportion（比例）、 Integral（积分）、Differential（微分）的首字母缩写。它是一种结合比例、积分和微分三个环节于一体的闭环控制算法，其模型如图所示：（引用：正点原子）



闲鱼：电子工程师的杂货铺

二、算法应用场景

PID 控制算法主要用于工业控制过程如：机器人、航空航天、汽车控制、电力系统、水处理等领域，在工业中常用在一下工况：

1）温度控制：如热处理炉、工业烘箱、恒温水槽等，市面上常用的温控器内置算法普遍为 PID 算法。

2）速度控制：如电机、液压马达、风扇等。

3）位置控制：如电梯、升降机等。

4）液位控制：如水泵、水箱、油罐等。

5）流量控制：如输送带、供水管道、喷嘴等，只要有关于电机调速的控制器，内置都有 PID 算法，如变频器。

6）其它控制领域：如飞行器姿态控制、汽车稳定性控制、电力调度等

闲鱼：电子工程师的杂货铺

三、算法讲解

1）比例环

比例环节可以成比例地反应控制系统的偏差信号，即输出与输入偏差成正比，可以用来减小系统的偏差。 此环节的公式如下：

$$u = Kp \cdot e$$

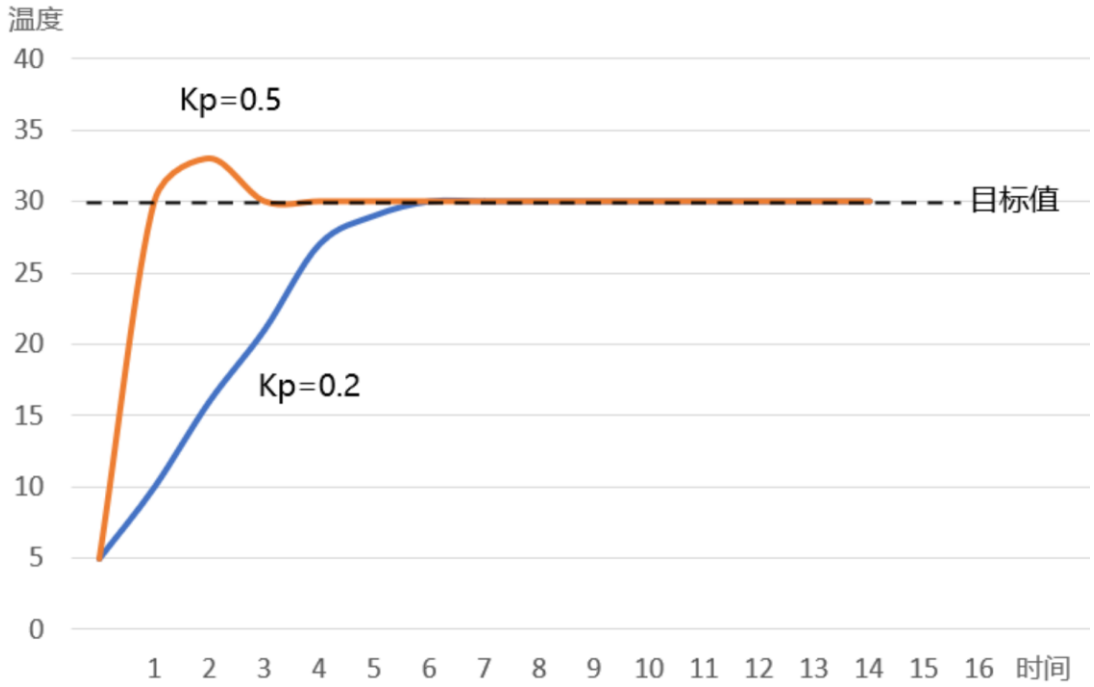
u ----- 输出

Kp ----- 比例系数

e ----- 偏差

假设我们现在需要调节棚内温度为 30℃，而实际温度为 10℃，此时的偏差 e=20，由比例环节的公式可知，当 e 确定时， Kp 越大则输出 u 越大，也就是温控系统的调节力度越大，这样就可以更快地达到目标温度；而当 Kp 确定时，偏差 e 越大则输出 u 越大。由此可见，在比例环节中，比例系数 Kp 和偏差 e 越大则系统消除偏差的时间越短，具体的曲线表现如图所示：

闲鱼：电子工程师的杂货铺

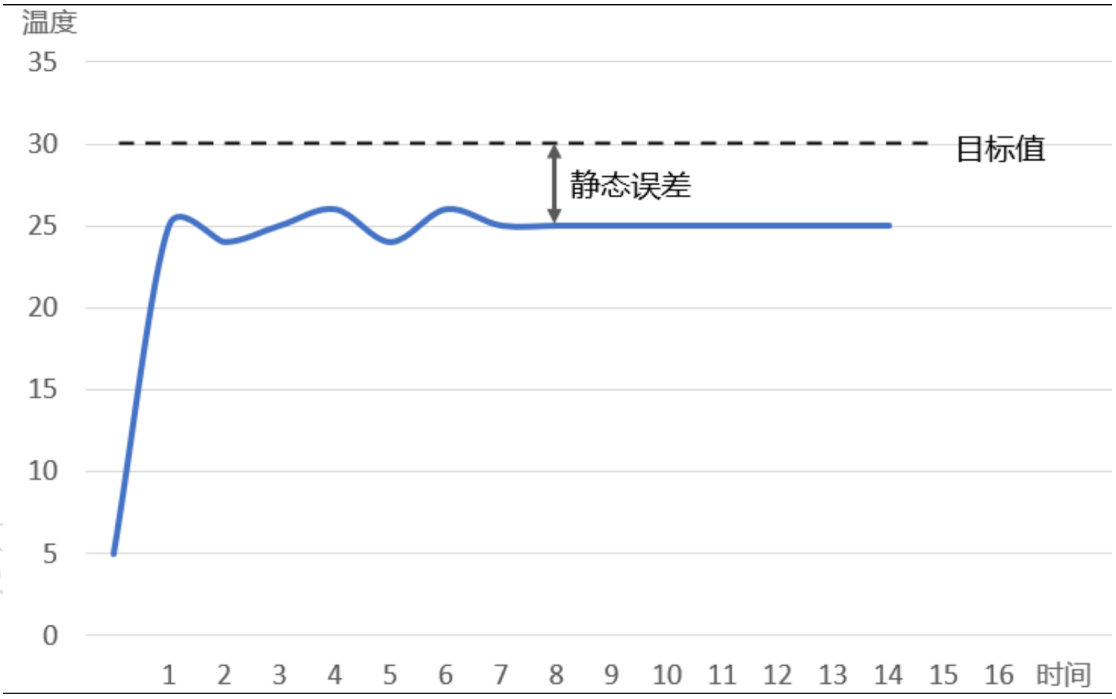


当 Kp 的值越大时，其对应的橙色曲线达到目标值的时间就越短，与此同时，橙色曲线出现了一定幅度的超调和振荡，这会使得系统的稳定性下降，因此， 我们在设置比例系数的时候，并不是越大越好，而是要兼顾消除偏差的时间以及整个系统的稳定性。然而，在实

际的应用中，如果仅有比例环节的控制，可能会给系统带来一个问题： 静态误差。静态误差是指系统控制过程趋于稳定时，目标值与实测值之间的偏差。我们以大棚温控实

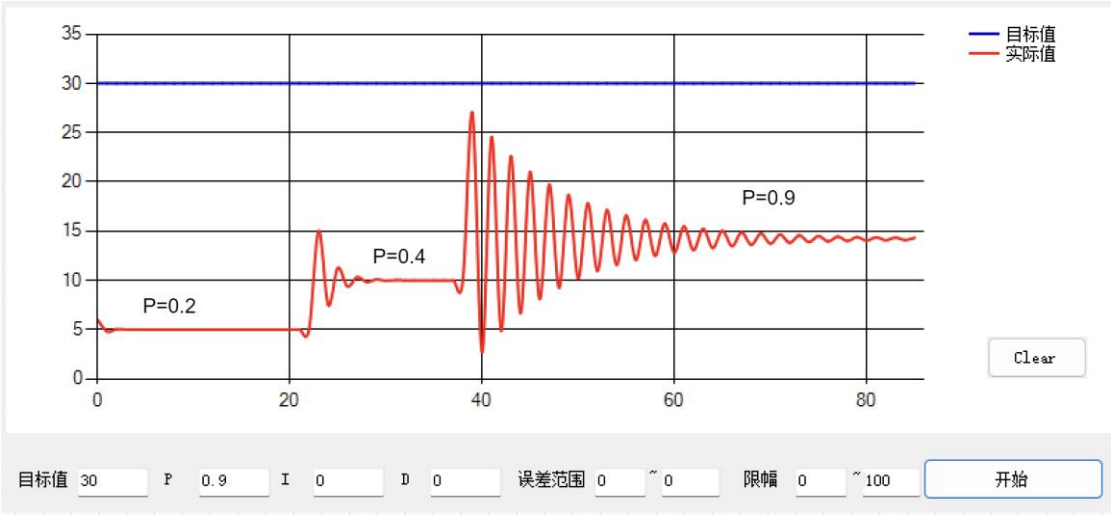
例来理解什么是静态误差：

假设我们现在需要调节棚内温度为 30℃，而实际温度为 25℃，此时偏差 e=5， Kp 为固定值，如果此时的输出可以让大棚在半个小时之内升温 5℃，而外部的温差可以让大棚在半个小时之内降温 5℃，也就是说，输出 u 的作用刚好被外部影响抵消了，这就使得偏差会一直存在，此时的曲线如图所示



大家可能会想到，我们可以通过增大 Kp 来增大输出，以此消除偏差。在实际应用中，此方法的局限性很大，因为我们不能确定偏差的大小，它是在实时变化的，如果我们把 Kp 设置得太大，就会引入超调和振荡，让整个系统的稳定性变差。因此，为了消除静态误差，我们引入了积分环节。（以上引用正点原子）

关于比例环节，我们实际使用中并没有理论那么正常，如下仿真图：



K 值越小，静态误差越大，系统反应越迟钝；K 值越大，静态误差越小，系统越灵敏，但是系统稳定性越差，K=0.9 时，系统长时间震荡。

2）积分环

积分环节可以对偏差 e 进行积分，只要存在偏差，积分环节就会不断起作用，主要用于消除静态误差，提高系统的无差度。 引入积分环节后，比例+积分环节的公式如下：

$$u = Kp \cdot e + Ki \cdot \sum e$$

u ----- 输出

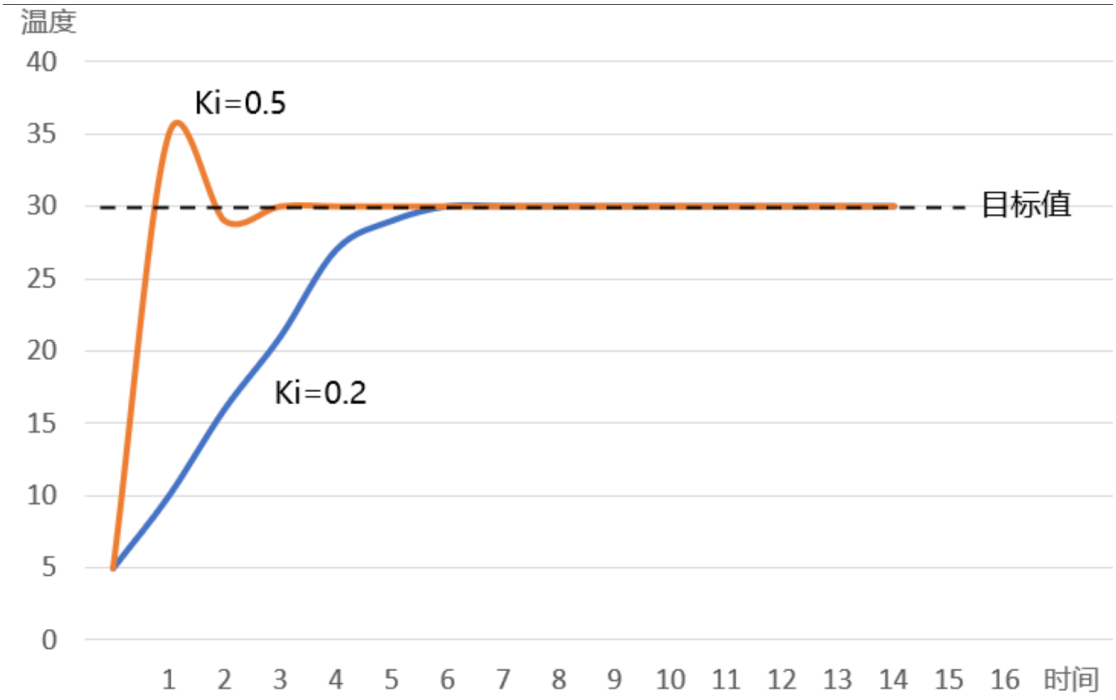
e ----- 偏差

Σ e ----- 累计偏差

Kp ----- 比例系数

Ki ----- 积分系数

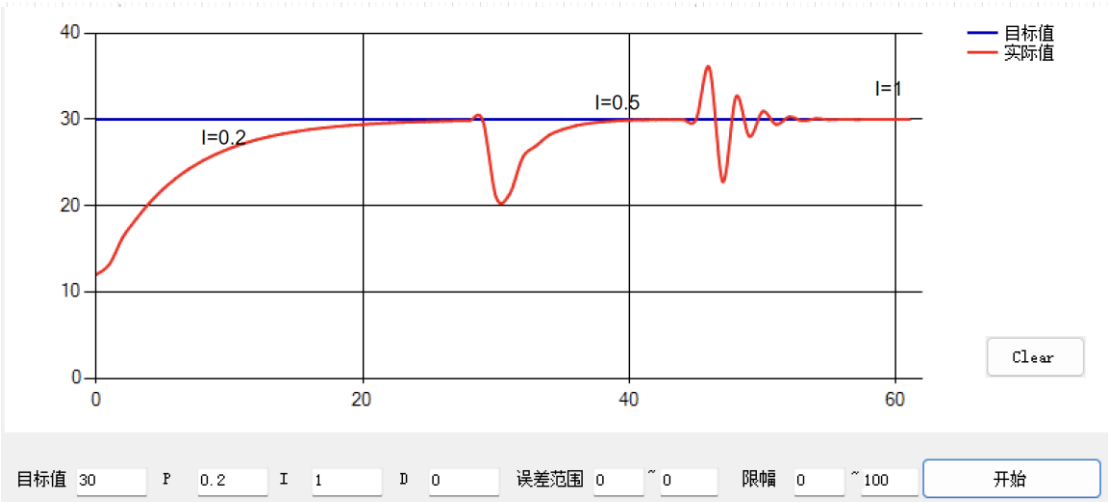
假设现在温控系统的比例环节作用被抵消， 存在静态误差 5℃，此时偏差存在，积分环节会一直累计偏差，以此增大输出，从而消除静态误差。从上述公式中可以得知，当积分系数 Ki 或者累计偏差越大时，输出就越大，系统消除静态误差的时间就越短，具体的曲线表现如图所示



当 Ki 的值越大时，其对应的橙色曲线达到目标值的时间就越短，与此同时，橙色曲线出现了一定幅度的超调和振荡，这会使得系统的稳定性下降，因此， 我们在设置积分系数的时候，并不是越大越好，而是要兼顾消除静态误差的时间以及整个系统的稳定性。我们前面有说过，只要系统还存在偏差，积分环节就会不断地累计偏差。当系统偏差为 0 的时候，说明已经达到目标值，此时的累计偏差不再变化，但是积分环节依旧在发挥作用（此时往往作用最大），这就很容易产生超调的现象了。因此，我们需要引入微分环节，提前减弱输出，抑制超调的发生（以上引用正点原子）

闲鱼：电子工程师的杂货铺

对于积分模拟图如下：



当 I 越小是，系统达到目标值的时间越长；当 I 越大时，系统达到目标值的时间短，但是会导致系统在稳定区间波动，出现超调现象，如 I=0.9 时。

3）微分环

微分环节可以反应偏差量的变化趋势，根据偏差的变化量提前作出相应控制，减小超调，克服振荡。 引入微分环节后，比例+积分+微分环节的公式如下：

$$u_k = Kp \cdot e_k + Ki\sum_{j=0}^{k} e_j + Kd(e_k - e_{k-1})$$

uk ---------- 第 k 次输出

Kp ---------- 比例系数

Ki ---------- 积分系数

Kd ---------- 微分系数

ek ---------- 第 k 次偏差

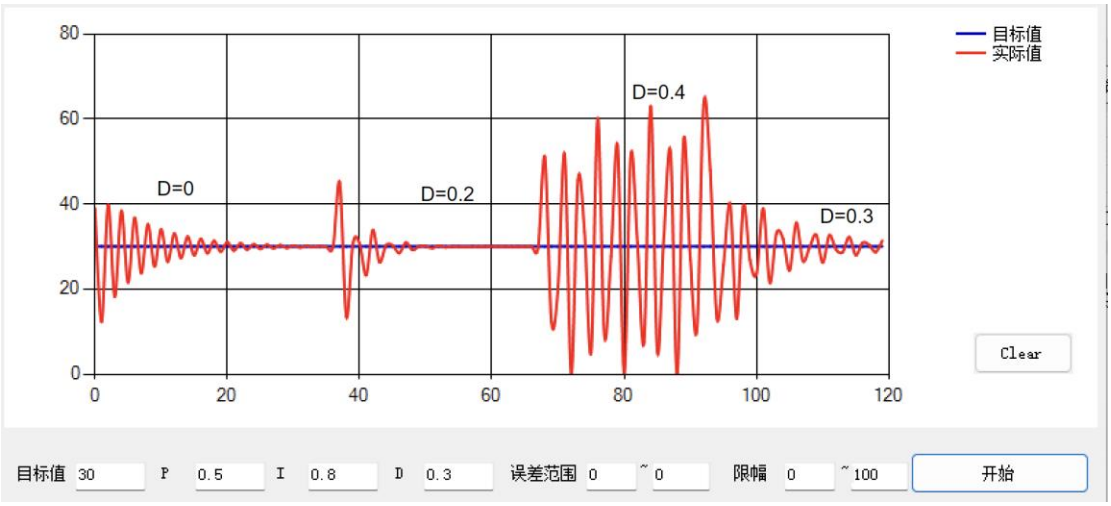
ek−1 ---------- 第 k-1 次偏差

∑*k j*=0 *ej* ---------- 累计 k 次偏差

公式是 PID 离散公式之一，除了离散公式之外， PID 还有连续的公式。（以上引用正点原子）

闲鱼：电子工程师的杂货铺

使用软件进行仿真，可以看出，当 D=0 时，系统超调震荡时间较长，当 D=0.2 是，系统能快速稳定下来，但是当 D 更大时，系统又重新不稳定起来。



所以，PID 算法的参数对系统的稳定性至关重要。下面介绍下常用的参数整定方法。

注意：PID 算法细分很多，但必须存在比例环

1） 比例环

2） 比例环+积分环

3） 比例环+微分环

4）比例环+积分环+微分环

4）PID 算法

1）位置式 PID 公式

$$u_k = Kp \cdot e_k + Ki\sum_{j=0}^{k} e_j + Kd(e_k - e_{k-1})$$

uk ---------- 第 k 次输出

Kp ---------- 比例系数

Ki ---------- 积分系数

Kd ---------- 微分系数

ek ---------- 第 k 次偏差

$ek-1$ ---------- 第 k-1 次偏差

$\sum k\ j=0\ ej$ ---------- 累计 k 次偏差

闲鱼：电子工程师的杂货铺

我们在上一小节中得到的 PID 离散公式称为位置式或全量式 PID 公式，该公式的计算需要全部控制量参与，它的每一次输出都和过去的状态有关。

2）增量式 PID 公式

$$\Delta u_k = Kp(e_k - e_{k-1}) + Ki \cdot e_k + Kd(e_k - 2e_{k-1} + e_{k-2})$$

增量式 PID 的计算并不需要一直累计偏差，它的输出与近三次的偏差有很大关系。

注意： 增量式 PID 公式输出的只是控制量的增量。

3）两种 PID 公式的优缺点

① 位置式：

优点：位置式 PID 是一种非递推式算法，带有积分作用，适用于不带积分部件的对象。

缺点：全量计算，计算错误影响很大；需要对偏差进行累加，运算量大。

② 增量式：

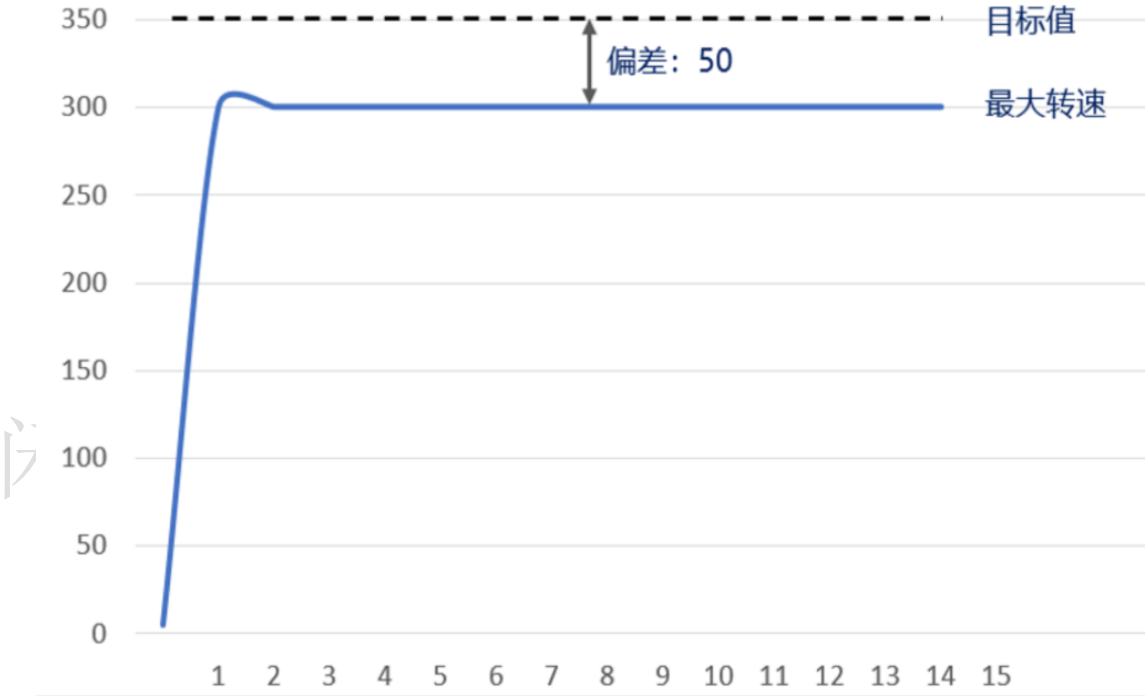
优点：只输出增量，计算错误影响小；不需要累计偏差，运算量少，实时性相对较好。

缺点：积分截断效应大，有稳态误差。

4）积分饱和

在位置式 PID 中，如果系统长时间无法达到目标值，累计偏差（积分）就会变得很大，此时系统的响应就很慢了。 我们以一个实例来分析这个问题：

假设某个电机能达到的最大速度为 300RPM，而我们设置了目标速度为 350RPM，这明显是一个不合理的目标值，此时 PID 控制系统的曲线如图所示：



由于系统长时间无法达到目标值，累计偏差（积分）会变得越来越大，逐渐达到深度饱和的状态，此时我们再设置一个合理范围的目标速度（例如 200RPM），系统就没有办法在短时间内响应了。

为了避免位置式 PID 中可能出现的积分饱和问题，可采取以下措施：

① 优化 PID 曲线，系统越快达到目标值，累计的偏差就越小；

② 限制目标值调节范围，规避可以预见的偏差；

③ 进行积分限幅，在调整好 PID 系数之后，根据实际系统来选择限幅范围。

（以上引用正点原子）

四、参数整定方法

1）试凑法

1）内容：结合系统的具体情况以及经验，先试凑几组合理的 PID 系数，同时需要观察系统的曲线变化，确定每一个系数对于整个系统曲线的大致影响，然后再根据具体的曲线进行调整。

2）调节思路：

① 先是比例（P），再积分（I），最后是微分（D）；

② 按纯比例系统整定比例系数，使其得到比较理想的调节过程曲线，然后再把比例系数缩小 1.2 倍左右，将积分系数从小到大改变，使其得到较好的调节过程曲线；

③ 在这个积分系数下重新改变比例系数，再看调节过程曲线有无改善；

④ 如有改善，可将原整定的比例系数减少，改变积分系数，这样多次的反复，就可得到合适的比例系数和积分系数；

⑤ 如果存在外界的干扰， 系统的稳定性不好， 可把比例、积分系数适当减小， 使系统足够稳定；

闲鱼：电子工程师的杂货铺

⑥ 如果系统存在小幅度超调， 可以将整定好的比例系数和积分系数适当减小， 增大微分系数，以得到超调量最小、调节作用时间最短的系统曲线；

2）临界比例法

1）内容： 在闭环的控制系统里，将调节器置于纯比例作用下，从小到大逐渐调节比例系数， 直到系统曲线出现等幅振荡，再根据经验公式计算参数。

2）调节思路：

① 将积分、 微分系数置零，比例度取适当值，平衡操作一段时间， 使控制系统按纯比例作用的方式投入运行。

② 慢慢地增大比例系数，细心观察曲线的变化情况。如果控制过程的曲线波动是衰减的，则把比例系数继续增大；如果曲线波动是发散的，则应把比例系数减小，直至曲线波动呈等幅振荡， 此时记下临界比例系数 δK 和临界振荡周期 Tk 的值。

③ 根据记下的比例系数和周期，采用经验公式，计算调节器的参数。

参与控制的环节	Kp	Ki	Kd
P	δK/2	0	0
PI	δK/2.2	Kp/（0.833• Tk）	0
PID	δK/1.7	Kp/（0.5• Tk）	0.125• Tk• Kp

临界比例法参数表

3）一般调节法

1）内容：这种方法针对一般的 PID 控制系统所以称之为一般调节法。

2）调节思路：

① 首先将积分、微分系数置零， 使系统为纯比例控制。 控制对象的值设定为系统允许的最大值的 60%~70%， 接着逐渐增大比例系数，直至系统出现振荡；此时再逐渐减小比例系数，直至系统振荡消失， 然后记录此时的比例系数， 并设定系统的比例系数为当前值的 60%~70%。

② 确定比例系数后，设定一个较小的积分系数，然后逐渐增大积分系数，直至系统出现振荡；此时在逐渐减小积分系数， 直至系统振荡消失，然后记录此时的积分系数，并设定系统的积分系数为当前值的 55%~65%。

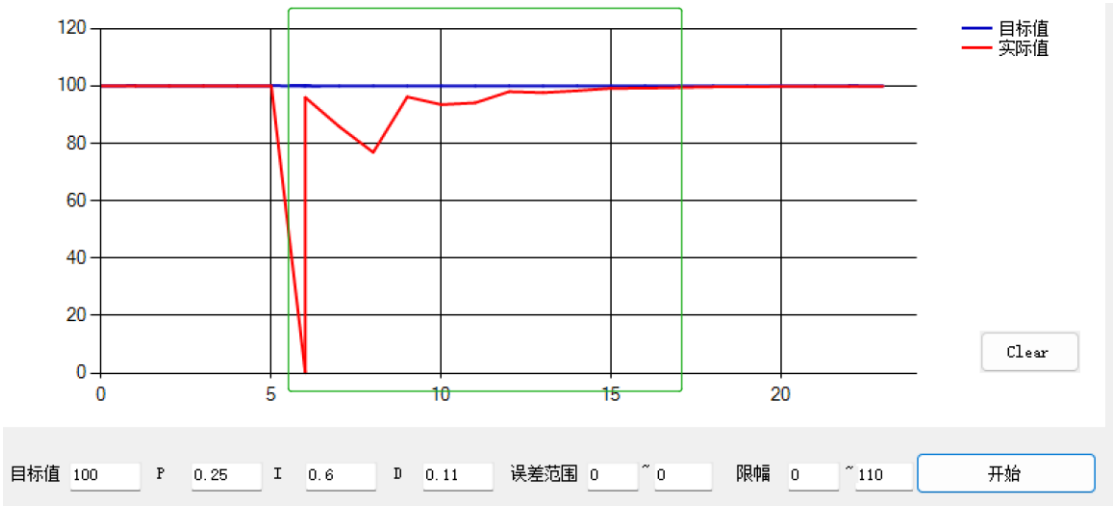
③ 微分系数一般不用设定，为 0 即可。若系统出现小幅度振荡， 并且通过 PI 环节无法优化，这可以采用与确定比例、积分系数相同的方法， 微分系数取系统不振荡时的 30%左右。

④ 系统空载、带载联调，再对 PID 参数进行微调，直至满足要求。

（以上引用正点原子）

闲鱼：电子工程师的杂货铺

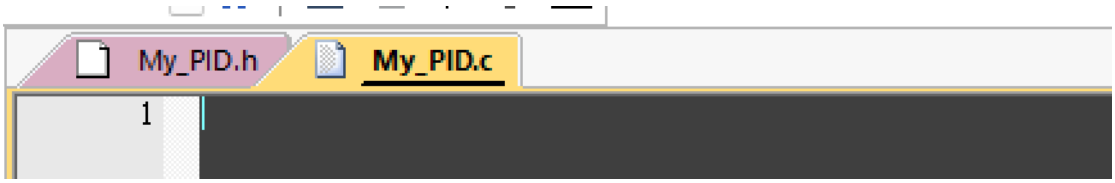
通过以上方式，我们可以尽快的找到合适的参数：



从图中可以看出，系统在上升阶段内未出现超调显现，且系统在 10 个调整周期内达到了稳态，并一直保持着。

五、软件编写

1）创建.c、.h 文件，并添加路径



2） 创建 PID 数据结构体

```
struct MY_PID_Type
{
  volatile double SetPoint;/*目标值*/
  volatile double OutPut;/*输出值*/
           double PID_P;
           double PID_I;
           double PID_D;
  volatile double Error;/*当前次误差*/
  volatile double Error_Last;/*上一次误差*/
  volatile double Error_Prev;/*上两次误差*/
  volatile double Sun_Error;/*累计误差*/
};
```

3）算法初始化函数

```
/*********************************************************************************************************
 * @name:MY_PID_Init
 * @fun: PID结构体初始化
 * @input:MY_PID_Type 结构体  P:比例  I: 积分  D 微分
 * @retv: 正常: 0  异常: 负数
 * @annot:
*********************************************************************************************************/
int MY_PID_Init(struct MY_PID_Type *pid,double P,double I,double D)
{
  if(pid == NULL)
    goto _State;
  if(P == 0)
    goto _State;
  pid->PID_P = P;
  pid->PID_I = I;
  pid->PID_D = D;
  pid->OutPut = 0;
  pid->Error = 0;
  pid->Error_Last = 0;
  pid->Error_Prev = 0;
  pid->Sun_Error  = 0;
  return 0;

_State:
  return -1;
}
```

3） 算法实现（增量式）

```
/*********************************************************************************************************
 * @name:MY_PID_Increment
 * @fun: 增量式PID计算
 * @input:MY_PID_Type 结构体  Value: 当前值   Out_Max: 输出限幅最大值  Out_Min: 输出限幅最小值
 * @retv: 输出值
 * @annot:
*********************************************************************************************************/
double MY_PID_Increment(struct MY_PID_Type *pid,double Value,double Out_Max,double Out_Min)
{
  pid->Error = pid->SetPoint - Value;/*计算当前次误差*/
  pid->OutPut +=  (pid->PID_P * (pid->Error - pid->Error_Last)) + /*比例环*/
                  (pid->PID_I * pid->Error) + /*积分环*/
                  (pid->PID_D * (pid->Error - 2*pid->Error_Last + pid->Error_Prev));/*微分环*/
  pid->Error_Prev = pid->Error_Last; /*误差存储*/
  pid->Error_Last = pid->Error;
  if(pid->OutPut > Out_Max)
    pid->OutPut = Out_Max;
  else if(pid->OutPut < Out_Min)
    pid->OutPut = Out_Min;
  return pid->OutPut;
}
```

6）算法实现（位置式）

```
/*************************************************************************************************
 * @name:MY_PID_Location
 * @fun: 位置式PID计算
 * @input:MY_PID_Type 结构体  Value：当前值   Out_Max：输出限幅最大值  Out_Min：输出限幅最小值
 * @retv: 输出值
 * @annot:
*************************************************************************************************/
double MY_PID_Location(struct MY_PID_Type *pid,double Value,double Out_Max,double Out_Min)
{
  pid->Error = pid->SetPoint - Value;/*计算当前次误差*/
  pid->Sun_Error += pid->Error;/*计算累计误差*/

  pid->OutPut =   (pid->PID_P * pid->Error) + /*比例环*/
                  (pid->PID_I * pid->Sun_Error) + /*积分环*/
                  (pid->PID_D * (pid->Error - pid->Error_Last));/*微分环*/
  pid->Error_Prev = pid->Error_Last; /*误差存储*/
  pid->Error_Last = pid->Error;
  if(pid->OutPut > Out_Max)
    pid->OutPut = Out_Max;
  else if(pid->OutPut < Out_Min)
    pid->OutPut = Out_Min;
  return pid->OutPut;
}
```

在实际的使用中，反馈值往往并不能直接作为算法的输入值，因为一旦反馈值出现一次较大的异常，输出值将会大幅度的波动，这某些工况是不允许的，所以需要怼反馈值进行滤波处理，常见的滤波算法有：

1）中值滤波

2）均值滤波

3）限幅滤波

4）平滑均值

5）平滑中值

6）卡尔曼滤波

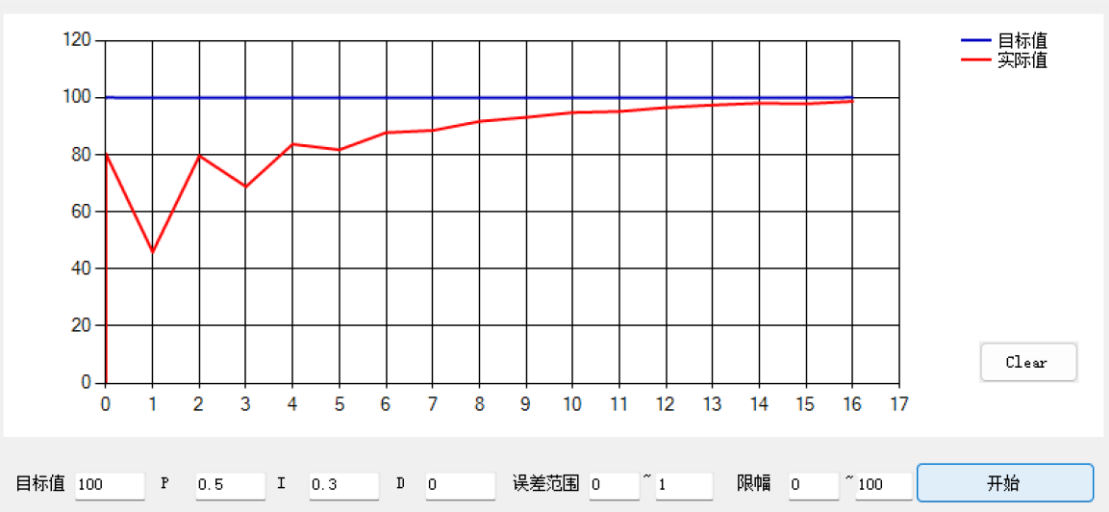
7）一阶滞后滤波

等等。

闲鱼：电子工程师的杂货铺

六、仿真软件使用

仿真软件目前只集成了增量式 PID 算法。



仿真软件设置有：

1） 目标值

2） 比例值 P

3） 积分制 I

4） 为分值 D

5） 误差范围（随机误差生成，模拟真实系统）

6） 限幅范围 （最大值与最小值）

注意以上参数必须是非负数，可以是小数，也可以是整数。

闲鱼：电子工程师的杂货铺

若文档或代码有错误的地方，还请不吝赐教，文档和代码均是自己写的，引用部分已标注清楚，若有侵权请联系。